```
import os
import time
import logging
from datetime import datetime
from binance.client import Client
from binance.exceptions import BinanceAPIException
import pandas as pd
import numpy as np

# ============================================================
# ⚙️  設定區（只需修改這裡）
# ============================================================
API_KEY           = os.environ.get("BINANCE_API_KEY", "你的API_KEY")
API_SECRET        = os.environ.get("BINANCE_SECRET",  "你的API_SECRET")

SYMBOL            = "BTCUSDT"
TRADE_USDT        = 10
INTERVAL          = Client.KLINE_INTERVAL_5MINUTE
STOP_LOSS_PCT     = 0.05          # 單筆止損 5%
DAILY_LOSS_LIMIT  = 0.05          # 當日止損 5%
SLEEP_SEC         = 300           # 每 5 分鐘檢查一次
MIN_SIGNALS       = 3             # 5 個指標符合 3 個以上才交易

USE_TESTNET       = True          # True=測試網 | False=真實交易

# ── 指標參數 ──────────────────────────────────────────
MA_SHORT          = 7
MA_LONG           = 25
RSI_PERIOD        = 14
RSI_OVERSOLD      = 30
RSI_OVERBOUGHT    = 70
MACD_FAST         = 12
MACD_SLOW         = 26
MACD_SIGNAL       = 9
BB_PERIOD         = 20
BB_STD            = 2
VOLUME_MA_PERIOD  = 20
VOLUME_MULT       = 1.5

# ============================================================
# 📋 Log 設定
# ============================================================
logging.basicConfig(
    level=logging.INFO,
```

```
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler("bot.log", encoding="utf-8"),
        logging.StreamHandler()
    ]
)
log = logging.getLogger(__name__)

# ============================================================
# 🔌 連線 Binance
# ============================================================
if USE_TESTNET:
    client = Client(API_KEY, API_SECRET, testnet=True)
    log.info("✅ 已連線到 Binance 測試網")
else:
    client = Client(API_KEY, API_SECRET)
    log.info("✅ 已連線到 Binance 真實交易")

# ============================================================
# 📊 狀態變數
# ============================================================
buy_price         = None
daily_start_usdt  = None
today_date        = None
is_paused_today   = False

# ============================================================
# 📥 取得 K 線資料
# ============================================================
def get_klines_df(symbol, interval, limit=100):
    klines = client.get_klines(symbol=symbol, interval=interval, limit=limit)
    df = pd.DataFrame(klines, columns=[
        "open_time","open","high","low","close","volume",
        "close_time","qav","trades","tbbav","tbqav","ignore"
    ])
    for col in ["open","high","low","close","volume"]:
        df[col] = df[col].astype(float)
    return df

# ============================================================
# 📈 指標計算
# ============================================================
def calc_rsi(series, period):
    delta = series.diff()
    gain  = delta.clip(lower=0).rolling(period).mean()
    loss  = (-delta.clip(upper=0)).rolling(period).mean()
    rs    = gain / loss
```

```
    return 100 - (100 / (1 + rs))

def calc_macd(series, fast, slow, signal):
    ema_fast    = series.ewm(span=fast, adjust=False).mean()
    ema_slow    = series.ewm(span=slow, adjust=False).mean()
    macd_line   = ema_fast - ema_slow
    signal_line = macd_line.ewm(span=signal, adjust=False).mean()
    return macd_line, signal_line

def calc_bb(series, period, std_mult):
    ma    = series.rolling(period).mean()
    std   = series.rolling(period).std()
    upper = ma + std_mult * std
    lower = ma - std_mult * std
    return upper, ma, lower

# ============================================================
# 🕯️ K 線形態偵測
# ============================================================
def detect_candle_pattern(df):
    o = df["open"].values
    h = df["high"].values
    l = df["low"].values
    c = df["close"].values
    bullish = False
    bearish = False

    body       = abs(c[-1] - o[-1])
    lower_wick = min(c[-1], o[-1]) - l[-1]
    upper_wick = h[-1] - max(c[-1], o[-1])

    # 錘子線
    if body > 0 and lower_wick >= 2 * body and upper_wick <= 0.3 * body and c[-1]
        bullish = True
        log.info("🕯️ 形態：錘子線（看漲）")

    # 多頭吞噬
    if c[-2] < o[-2] and c[-1] > o[-1] and o[-1] <= c[-2] and c[-1] >= o[-2]:
        bullish = True
        log.info("🕯️ 形態：多頭吞噬（看漲）")

    # 流星線
    if body > 0 and upper_wick >= 2 * body and lower_wick <= 0.3 * body and c[-1]
        bearish = True
        log.info("🕯️ 形態：流星線（看跌）")

    # 空頭吞噬
```

```
    if c[-2] > o[-2] and c[-1] < o[-1] and o[-1] >= c[-2] and c[-1] <= o[-2]:
        bearish = True
        log.info("🕯️  形態：空頭吞噬（看跌）")

    return bullish, bearish

# ============================================================
# 🔎 信號評分（5 個指標，每個最多 1 分）
# ============================================================
def evaluate_signals(df):
    close  = df["close"]
    volume = df["volume"]
    buy_score  = 0
    sell_score = 0
    detail     = []

    # 1. MA 均線交叉
    ma_s = close.rolling(MA_SHORT).mean()
    ma_l = close.rolling(MA_LONG).mean()
    if ma_s.iloc[-2] < ma_l.iloc[-2] and ma_s.iloc[-1] > ma_l.iloc[-1]:
        buy_score += 1
        detail.append("✅ MA 黃金交叉")
    elif ma_s.iloc[-2] > ma_l.iloc[-2] and ma_s.iloc[-1] < ma_l.iloc[-1]:
        sell_score += 1
        detail.append("❌ MA 死亡交叉")
    else:
        detail.append("⬜ MA 無信號")

    # 2. RSI
    rsi     = calc_rsi(close, RSI_PERIOD)
    rsi_val = rsi.iloc[-1]
    if rsi_val < RSI_OVERSOLD:
        buy_score += 1
        detail.append(f"✅ RSI 超賣 ({rsi_val:.1f})")
    elif rsi_val > RSI_OVERBOUGHT:
        sell_score += 1
        detail.append(f"❌ RSI 超買 ({rsi_val:.1f})")
    else:
        detail.append(f"⬜ RSI 中性 ({rsi_val:.1f})")

    # 3. MACD
    macd_line, signal_line = calc_macd(close, MACD_FAST, MACD_SLOW, MACD_SIGNAL)
    if macd_line.iloc[-2] < signal_line.iloc[-2] and macd_line.iloc[-1] > signal_
        buy_score += 1
        detail.append("✅ MACD 上穿信號線")
    elif macd_line.iloc[-2] > signal_line.iloc[-2] and macd_line.iloc[-1] < signa
        sell_score += 1
```

```
        detail.append("❌ MACD 下穿信號線")
    else:
        detail.append("⬜ MACD 無交叉")

    # 4. 布林通道
    bb_upper, bb_mid, bb_lower = calc_bb(close, BB_PERIOD, BB_STD)
    price_now = close.iloc[-1]
    if price_now <= bb_lower.iloc[-1]:
        buy_score += 1
        detail.append("✅ 布林下軌突破（超賣區）")
    elif price_now >= bb_upper.iloc[-1]:
        sell_score += 1
        detail.append("❌ 布林上軌突破（超買區）")
    else:
        detail.append("⬜ 布林通道中性")

    # 5. 成交量 + K 線形態
    vol_ma          = volume.rolling(VOLUME_MA_PERIOD).mean().iloc[-1]
    vol_now         = volume.iloc[-1]
    high_volume     = vol_now >= vol_ma * VOLUME_MULT
    bullish_candle, bearish_candle = detect_candle_pattern(df)

    if high_volume and bullish_candle:
        buy_score += 1
        detail.append("✅ 量能放大 + 看漲K線形態")
    elif high_volume and bearish_candle:
        sell_score += 1
        detail.append("❌ 量能放大 + 看跌K線形態")
    elif high_volume:
        detail.append("⬜ 量能放大但形態不明確")
    else:
        detail.append(f"⬜ 成交量普通 ({vol_now:.0f} vs 均量 {vol_ma:.0f})")

    return buy_score, sell_score, detail

# ============================================================
# 💰 查詢餘額
# ============================================================
def get_balance(asset):
    try:
        bal = client.get_asset_balance(asset=asset)
        return float(bal["free"]) if bal else 0.0
    except Exception as e:
        log.error(f"查詢餘額失敗: {e}")
        return 0.0

def get_total_usdt(current_price):
```

```
    usdt = get_balance("USDT")
    btc  = get_balance("BTC")
    return usdt + btc * current_price

# ============================================================
# 📈 買入 / 📉 賣出
# ============================================================
def buy(usdt_amount, price):
    qty = round(usdt_amount / price, 5)
    try:
        order = client.order_market_buy(symbol=SYMBOL, quantity=qty)
        log.info(f"🟢 買入 {qty} BTC @ {price:.2f} | 訂單: {order['orderId']}")
        return price
    except BinanceAPIException as e:
        log.error(f"買入失敗: {e}")
        return None

def sell(qty, reason="賣出"):
    try:
        order = client.order_market_sell(symbol=SYMBOL, quantity=round(qty, 5))
        log.info(f"🔴 {reason} | 賣出 {qty:.5f} BTC | 訂單: {order['orderId']}")
    except BinanceAPIException as e:
        log.error(f"賣出失敗: {e}")

# ============================================================
# 🤖 主程式
# ============================================================
def main():
    global buy_price, daily_start_usdt, today_date, is_paused_today

    log.info("🚀 升級版交易機器人啟動！")
    log.info(f"   指標：MA + RSI + MACD + 布林通道 + 量能/K線形態")
    log.info(f"   觸發條件：5 個指標中符合 {MIN_SIGNALS} 個以上才交易")
    log.info(f"   單筆止損：{STOP_LOSS_PCT*100}% | 當日止損：{DAILY_LOSS_LIMIT*100}%

    while True:
        try:
            now   = datetime.now()
            today = now.date()

            # 每天重置
            if today != today_date:
                today_date      = today
                is_paused_today = False
                df_init         = get_klines_df(SYMBOL, INTERVAL)
                daily_start_usdt= get_total_usdt(df_init["close"].iloc[-1])
                log.info(f"📅 新的一天 {today} | 起始資產：{daily_start_usdt:.2f} US
```

```
# 取得市場資料
df            = get_klines_df(SYMBOL, INTERVAL, limit=100)
current_price = df["close"].iloc[-1]
usdt_balance  = get_balance("USDT")
btc_balance   = get_balance("BTC")
total_now     = get_total_usdt(current_price)
daily_loss_pct= (daily_start_usdt - total_now) / daily_start_usdt

# 信號評分
buy_score, sell_score, detail = evaluate_signals(df)

log.info(
    f"💹 {current_price:.2f} USDT | "
    f"買入信號: {buy_score}/5 | 賣出信號: {sell_score}/5 | "
    f"資產: {total_now:.2f} | 當日損益: {-daily_loss_pct*100:+.2f}%"
)
for d in detail:
    log.info(f"   {d}")

# 當日虧損超限
if daily_loss_pct >= DAILY_LOSS_LIMIT and not is_paused_today:
    is_paused_today = True
    log.warning(f"⛔ 當日虧損達 {daily_loss_pct*100:.1f}%，今日交易暫停！
    if btc_balance > 0.0001:
        sell(btc_balance, reason="當日止損強制賣出")
        buy_price = None

if is_paused_today:
    log.info("⏸️  今日已暫停，等待明天重置...")
    time.sleep(SLEEP_SEC)
    continue

# 買入判斷
if buy_score >= MIN_SIGNALS and btc_balance <= 0.0001:
    if usdt_balance >= TRADE_USDT:
        log.info(f"📡 買入條件達成（{buy_score}/5 個指標符合）")
        buy_price = buy(TRADE_USDT, current_price)
    else:
        log.warning(f"⚠️  USDT 不足（{usdt_balance:.2f}），跳過買入")

# 賣出判斷
elif sell_score >= MIN_SIGNALS and btc_balance > 0.0001:
    log.info(f"📡 賣出條件達成（{sell_score}/5 個指標符合）")
    sell(btc_balance, reason=f"策略賣出({sell_score}/5)")
    buy_price = None
```

```
            # 單筆止損
            if buy_price and btc_balance > 0.0001:
                loss_pct = (buy_price - current_price) / buy_price
                if loss_pct >= STOP_LOSS_PCT:
                    log.warning(f"🛑 單筆止損！虧損 {loss_pct*100:.1f}%")
                    sell(btc_balance, reason="單筆止損賣出")
                    buy_price = None

        except Exception as e:
            log.error(f"❌ 發生錯誤: {e}")

        time.sleep(SLEEP_SEC)

if __name__ == "__main__":
    main()
```